DEVOPS ENGINEERING MASTERY

NETWORKING & INTERNET INFRASTRUCTURE

নেটওয়ার্কিং, Internet Protocols ও Production Network Engineering

VOLUME 02 • FIRST EDITION

```
nahid@ubuntu-server:~$ ip -br addr
eth0   UP   192.168.56.10/24
nahid@ubuntu-server:~$ dig +short example.com
93.184.216.34
nahid@ubuntu-server:~$ curl -I https://example.com
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs + 40 Subnetting Questions

Practical Lab & Assignment Workbook

DEVOPS ENGINEERING MASTERY — Volume 02 (Companion Workbook). First Edition, 8 November 2026. Ubuntu Server 24.04 LTS. This workbook collects Appendix A (24 guided labs), Appendix B (15 independent assignments) and Appendix C (3 major projects) from the textbook, so you can work through them beside your virtual machine without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab command on a production network. Solutions and marking guidance are in the separate Quiz & Answer Guide — attempt each assignment before looking.

পরিশিষ্ট A — Guided Labs (গাইডেড ল্যাব)

এই পরিশিষ্টের ২৪টি ল্যাব এই বইয়ের ব্যবহারিক হৃদয়। প্রতিটি ল্যাব একটি বাস্তব পরিস্থিতি থেকে শুরু হয়, একটি নিরাপদ বিচ্ছিন্ন ল্যাব নেটওয়ার্কে চলে, এবং যাচাই দিয়ে শেষ হয়। সব ল্যাবে ব্যবহৃত ঠিকানা (যেমন 192.168.56.0/24) শুধু উদাহরণের ল্যাব ঠিকানা — কোনো বাস্তব ব্যবহারকারীর নেটওয়ার্কের তথ্য নয়। নিজের নয় এমন কোনো নেটওয়ার্ক স্ক্যান বা আক্রমণ করবেন না; সব পরীক্ষা নিজের VM ও নিজের তৈরি সার্ভিসের ওপর করুন।

> **WARNING:** ল্যাবের কমান্ড শুধু আপনার নিজের VirtualBox VM বা বিচ্ছিন্ন ল্যাব নেটওয়ার্কে চালান। `tcpdump`, `nmap` বা firewall নিয়ম কখনও অফিস, বিশ্ববিদ্যালয় বা অন্যের নেটওয়ার্কে চালাবেন না। UFW চালু করার আগে SSH-এর নিয়ম যোগ করুন, নইলে নিজেই সার্ভারের বাইরে আটকে যাবেন।

Lab 01 — Interface ও IP Address চেনা: নিজের সার্ভারের পরিচয়পত্র

পরিস্থিতি: আপনি TechNova-র নতুন জুনিয়র ইঞ্জিনিয়ার। প্রথম দিন সার্ভারে লগইন করে জানতে হবে — এই মেশিনের interface কী কী, কোন IP-তে চলছে, gateway কোনটি।

উদ্দেশ্য: `ip` কমান্ড দিয়ে interface, address ও route পড়তে পারা।

পূর্বজ্ঞান: অধ্যায় ১, ৬, ৭। **টুল:** Ubuntu Server 24.04 VM। **প্রাথমিক অবস্থা:** VM চালু, NAT বা Host-only নেটওয়ার্কে যুক্ত।

ধাপ:

1. `ip -brief addr` চালান — প্রতিটি interface-এর সংক্ষিপ্ত তালিকা পাবেন। `ip` হলো আধুনিক Linux নেটওয়ার্ক টুল; পুরোনো `ifconfig` আর ডিফল্টভাবে থাকে না।
2. `ip addr show` চালান — প্রতিটি interface-এর MAC, IPv4/IPv6, broadcast ও state (UP/DOWN) দেখুন।
3. `ip route` চালান — default gateway কোনটি, তা লিখে রাখুন।
4. `ip link show` চালান — link-এর MTU ও state যাচাই করুন।

```
ip -brief addr
ip addr show
ip route
ip link show
```

উদাহরণ আউটপুট (illustrative):

```
lo               UNKNOWN        127.0.0.1/8
enp0s3           UP             192.168.56.10/24
default via 192.168.56.1 dev enp0s3 proto dhcp
```

যাচাই: `ip -brief addr`-এ অন্তত একটি UP interface ও একটি IPv4 ঠিকানা আছে; `ip route`-এ default লাইন আছে। **সাধারণ ব্যর্থতা:** interface DOWN থাকলে `sudo ip link set enp0s3 up` (নিজের VM-এই শুধু) চেষ্টা করুন; DHCP না পেলে VM-এর নেটওয়ার্ক সেটিংস দেখুন। **Cleanup:** কোনো পরিবর্তন না করলে কিছু মুছতে হবে না।

Lab 02 — Binary ও Subnetting হাতে-কলমে: কাগজ, তারপর টার্মিনাল

পরিস্থিতি: TechNova-র নেটওয়ার্ক পরিকল্পনায় 192.168.56.0/24 নেটওয়ার্ককে চারটি সমান subnet-এ ভাগ করতে হবে। প্রথমে কাগজে, তারপর `ipcalc` দিয়ে যাচাই।

উদ্দেশ্য: /24 থেকে /26 subnet-এর network, broadcast ও usable host হিসাব করা।

পূর্বজ্ঞান: অধ্যায় ৭, ৮। **টুল:** কাগজ-কলম, VM। **প্রাথমিক অবস্থা:** কোনো পরিবর্তন নেই।

1. কাগজে লিখুন: /26 মানে ২টি host bit ধার নেওয়া, ৪টি subnet, প্রতিটিতে ৬২টি usable host। প্রথম subnet: 192.168.56.0/26, broadcast 192.168.56.63।
2. VM-এ `sudo apt update && sudo apt install -y ipcalc` দিয়ে ipcalc ইনস্টল করুন।
3. `ipcalc 192.168.56.0/26` চালিয়ে নিজের হিসাবের সঙ্গে মেলান।
4. `ipcalc 192.168.56.64/26` ও `ipcalc 192.168.56.128/26` চালিয়ে বাকি দুটি subnet যাচাই করুন।

```
sudo apt update && sudo apt install -y ipcalc
ipcalc 192.168.56.0/26
ipcalc 192.168.56.64/26
ipcalc 10.0.0.0/22
```

উদাহরণ আউটপুট (illustrative): ipcalc দেখাবে Address, Netmask 255.255.255.192, Network, Broadcast 192.168.56.63, Hosts/Net: 62। **যাচাই:** আপনার কাগজের হিসাবের সঙ্গে ipcalc-এর Network/Broadcast/Hosts মিলেছে। **সাধারণ ভুল:** network ও broadcast ঠিকানাকে host ধরা; /26-এ ৬৪ নয়, ৬২টি usable host। **Cleanup:** ipcalc রেখে দিতে পারেন; শেখার টুল হিসেবে কাজে লাগবে।

Lab 03 — Routing Table পড়া ও Static Route যোগ করা

পরিস্থিতি: আপনার VM-এ দ্বিতীয় একটি নেটওয়ার্ক (192.168.99.0/24) যুক্ত হয়েছে, কিন্তু সেদিকে কোনো route নেই। Static route যোগ করে পৌঁছাতে হবে।

উদ্দেশ্য: routing table পড়া, static route যোগ ও মুছে ফেলা।

পূর্বজ্ঞান: অধ্যায় ১২। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** শুধু default route আছে।

1. `ip route show` দিয়ে বর্তমান table লিখে রাখুন।
2. `sudo ip route add 192.168.99.0/24 via 192.168.56.1 dev enp0s3` চালান — `via` মানে next-hop, `dev` মানে বেরোনোর interface।

3. `ip route show` দিয়ে নতুন লাইনটি যাচাই করুন।
4. `ping -c 2 192.168.99.1` চালান — উত্তরের আশা করবেন না, কারণ ওই নেটওয়ার্ক আসলে নেই; উদ্দেশ্য route lookup যাচাই, `ip route get 192.168.99.5` দিয়ে দেখুন কোন পথ বাছা হচ্ছে।
5. `sudo ip route del 192.168.99.0/24` দিয়ে route মুছে ফেলুন।

```
ip route show
sudo ip route add 192.168.99.0/24 via 192.168.56.1 dev enp0s3
ip route get 192.168.99.5
sudo ip route del 192.168.99.0/24
```

যাচাই: `ip route get` দেখায় 192.168.99.5-এর জন্য via 192.168.56.1; মুছে ফেলার পর লাইনটি আর নেই। **সাধারণ ব্যর্থতা:** ভুল gateway দিলে `Network is unreachable` আসে; `ip route` দিয়ে next-hop নিজের subnet-এ আছে কি না দেখুন। **Cleanup:** route মুছে ফেলা হয়েছে; reboot-এ এমন temporary route নিজে থেকেই মুছে যায়।

Lab 04 — TCP ও UDP সংযোগ: কোনটি কখন, নিজের হাতে দেখা

পরিস্থিতি: একটি web সার্ভিস TCP-তে চলে, একটি DNS-জাতীয় সার্ভিস UDP-তে। দুটি সংযোগের আচরণের পার্থক্য নিজে দেখতে হবে।

উদ্দেশ্য: `nc` (netcat) দিয়ে TCP listener ও UDP পরীক্ষা করা।

পূর্বজ্ঞান: অধ্যায় ১০, ১১। **টুল:** VM, `nc`। **প্রাথমিক অবস্থা:** কোনো listener নেই।

1. `sudo apt install -y netcat-openbsd` দিয়ে nc ইনস্টল করুন।
2. একটি টার্মিনালে `nc -l 8080` চালান — এটি TCP port 8080-এ শোনে।
3. আরেকটি টার্মিনালে `echo hello | nc 127.0.0.1 8080` চালান — প্রথম টার্মিনালে hello দেখা যাবে।
4. UDP-তে `nc -u -l 9090` ও `echo hi | nc -u 127.0.0.1 9090` দিয়ে একই পরীক্ষা করুন।
5. `ss -tlnp` দিয়ে দেখুন কোন TCP port শুনছে; UDP-র জন্য `ss -ulnp`।

```
sudo apt install -y netcat-openbsd
nc -l 8080
echo hello | nc 127.0.0.1 8080
ss -tlnp
ss -ulnp
```

যাচাই: TCP-তে বার্তা পৌঁছেছে; `ss`-এ 8080 LISTEN দেখা গেছে। **সাধারণ ব্যর্থতা:** port ইতিমধ্যে ব্যবহৃত হলে অন্য port (যেমন 8081) নিন; `-l` ছাড়া nc ক্লায়েন্ট হিসেবে চলে। **Cleanup:** Ctrl+C দিয়ে listener বন্ধ করুন; `ss -tlnp` দিয়ে নিশ্চিত হোন।

Lab 05 — Port Inspection: কোন সার্ভিস কোন পোর্টে শুনছে

পরিস্থিতি: TechNova-র সার্ভারে একটি সার্ভিস চালু হচ্ছে না বলে অভিযোগ। প্রথমে জানতে হবে — কোন port খোলা, কোন process সেখানে বাঁধা।

উদ্দেশ্য: `ss` দিয়ে listening port ও process শনাক্ত করা।

পূর্বজ্ঞান: অধ্যায় ১০, ২১। **টুল:** VM। **প্রাথমিক অবস্থা:** SSH চলছে।

1. `ss -tlnp` চালান — TCP, listening, সংখ্যায় port, process-সহ দেখাবে।
2. `ss -tlnp | grep 22` দিয়ে SSH (port 22) যাচাই করুন।
3. `ss -tulpn` দিয়ে TCP ও UDP দুটোই দেখুন।
4. `ss -tn state established` দিয়ে বর্তমান সক্রিয় সংযোগগুলো দেখুন।
5. একটি port-এ কিছু না শুনলে `curl -v http://127.0.0.1:8080/` চালিয়ে `Connection refused` বার্তাটি দেখুন — এটি প্রমাণ করে listener নেই, firewall নয়।

```
ss -tlnp
ss -tulpn
ss -tn state established
curl -v http://127.0.0.1:8080/
```

উদাহরণ আউটপুট (illustrative): `LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=812,fd=3))` — 0.0.0.0 মানে সব interface-এ শোনা হচ্ছে। **যাচাই:** port 22 LISTEN দেখা গেছে; `curl` বন্ধ port-এ refused দিয়েছে। **সাধারণ ভুল:** `Connection refused` আর `Connection timed out` গুলিয়ে ফেলা — প্রথমটি listener নেই, দ্বিতীয়টি পথে বাধা (firewall/routing)। **Cleanup:** কিছু বদলানো হয়নি।

Lab 06 — DNS Lookup: একটি নাম কীভাবে ঠিকানা হয়

পরিস্থিতি: ব্যবহারকারী বলছে example.com খুলছে না। DNS-ই কি দোষী, নাকি সার্ভার? `dig` দিয়ে ধাপে ধাপে দেখুন।

উদ্দেশ্য: `dig` দিয়ে A, AAAA, MX, NS রেকর্ড পড়া ও cache/TTL বোঝা।

পূর্বজ্ঞান: অধ্যায় ১৩। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `dig example.com` চালান — ANSWER SECTION-এ A রেকর্ড, QUERY TIME, SERVER দেখুন।
2. `dig example.com AAAA` ও `dig example.com MX` চালান।
3. `dig +trace example.com` চালান — root থেকে authoritative পর্যন্ত iterative যাত্রা দেখুন।
4. `dig example.com +short` দিয়ে শুধু ঠিকানাটি নিন।
5. `cat /etc/resolv.conf` দিয়ে আপনার resolver কোনটি, তা দেখুন।

```
dig example.com
dig example.com MX
dig +trace example.com
dig example.com +short
cat /etc/resolv.conf
```

যাচাই: A রেকর্ডে একটি IP ফিরেছে; `+trace`-এ root, TLD ও authoritative সার্ভারের ধাপ দেখা গেছে। **সাধারণ ব্যর্থতা:** `NXDOMAIN` মানে নামটি নেই; `SERVFAIL` মানে resolver সমস্যা — তখন `dig @8.8.8.8 example.com` দিয়ে অন্য resolver পরীক্ষা করুন (শুধু পরীক্ষার জন্য)। **Cleanup:** কিছু বদলানো হয়নি।

Lab 07 — DHCP, ARP ও ICMP: ল্যানের ভেতরের কথোপকথন

পরিস্থিতি: নতুন VM নেটওয়ার্কে এলো। DHCP থেকে ঠিকানা পেল কি না, ARP table-এ প্রতিবেশী কারা, ping আসলে কী করে — নিজে দেখুন।

উদ্দেশ্য: DHCP lease, ARP cache ও ICMP যাচাই করা।

পূর্বজ্ঞান: অধ্যায় ১৪। **টুল:** VM। **প্রাথমিক অবস্থা:** DHCP-তে ঠিকানা পাওয়া।

1. `ip addr show` দিয়ে DHCP-পাওয়া ঠিকানা দেখুন; `cat /var/lib/dhcp/dhclient.leases` থাকলে lease-এর বিবরণ দেখুন (ফাইল না থাকলে systemd-networkd-এর lease `networkctl status` দিয়ে দেখুন)।
2. `ping -c 4 192.168.56.1` চালান — gateway-তে ICMP echo পৌঁছেছে কি না দেখুন।
3. `ip neigh show` চালান — ARP cache-এ gateway-এর MAC ঠিকানা দেখা যাবে।
4. `ping -c 2 8.8.8.8` দিয়ে ইন্টারনেট পথ যাচাই করুন (ইন্টারনেট থাকলে)।
5. `traceroute 8.8.8.8` বা `tracepath 8.8.8.8` চালিয়ে hop-গুলো দেখুন (না থাকলে `sudo apt install -y traceroute`)।

```
ping -c 4 192.168.56.1
ip neigh show
ping -c 2 8.8.8.8
tracepath 8.8.8.8
```

যাচাই: ping-এ 0% packet loss; `ip neigh`-এ gateway-এর REACHABLE এন্ট্রি আছে। **সাধারণ ব্যর্থতা:** ping ব্যর্থ হলে প্রথমে `ip route` দিয়ে default gateway আছে কি না দেখুন; gateway-তে ping না গেলে VM-এর নেটওয়ার্ক মোড দেখুন। **Cleanup:** কিছু বদলানো হয়নি।

Lab 08 — HTTP Request হাতে-কলমে: curl দিয়ে ওয়েবের ভেতর দেখা

পরিস্থিতি: একটি ওয়েবসাইট 404 দিচ্ছে। ব্রাউজার শুধু পৃষ্ঠা দেখায়; `curl` দিয়ে আসল request/response দেখতে হবে।

উদ্দেশ্য: `curl -v` দিয়ে method, header, status code পড়া।

পূর্বজ্ঞান: অধ্যায় ১৫। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `curl -I https://example.com` চালান — শুধু response header দেখুন (HTTP/2 200, content-type)।
2. `curl -v https://example.com 2>&1 | head -30` চালিয়ে DNS, TLS ও header ধাপগুলো পড়ুন।
3. `curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com` দিয়ে status ও সময় মাপুন।
4. `curl -I https://example.com/nonexistent-page` দিয়ে 404 প্রতিক্রিয়া দেখুন।

```
curl -I https://example.com
curl -v https://example.com 2>&1 | head -30
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
```

যাচাই: 200 status পাওয়া গেছে; 404 পৃষ্ঠায় 404 দেখা গেছে; `-v`-এ TLS handshake লাইনগুলো দেখা গেছে। **সাধারণ ভুল:** `-I` মানে HEAD request — body আসে না, এটি স্বাভাবিক। **Cleanup:** কিছু বদলানো হয়নি।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 09 — TLS Certificate পরীক্ষা: openssl দিয়ে সার্টিফিকেট পড়া

পরিস্থিতি: একটি সাইটের certificate কবে মেয়াদ শেষ হবে, কে দিয়েছে, কোন নামের জন্য — ব্রাউজার না খুলে টার্মিনালেই যাচাই করতে হবে।

উদ্দেশ্য: `openssl s_client` দিয়ে certificate chain, মেয়াদ ও issuer পড়া।

পূর্বজ্ঞান: অধ্যায় ১৬। **টুল:** VM, openssl। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates` চালান — `-servername` SNI পাঠায়, নইলে ভুল certificate আসতে পারে।
2. `echo | openssl s_client -connect example.com:443 2>/dev/null | grep -E "Verify return code|Protocol"` দিয়ে validation ও TLS সংস্করণ দেখুন।
3. মেয়াদ শেষের তারিখ (notAfter) লিখে রাখুন — production-এ এই তারিখই monitoring-এর বিষয়।

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

যাচাই: subject-এ example.com, issuer-এ একটি পরিচিত CA, notAfter ভবিষ্যতের তারিখ। **সাধারণ ব্যর্থতা:** `Verify return code: 21` মানে self-signed বা অচেনা chain; `unable to get local issuer certificate` মানে intermediate certificate অনুপস্থিত। **নিরাপত্তা:** কখনও private key (`.key` ফাইল) কারও সঙ্গে শেয়ার করবেন না, Git-এ তুলবেন না। **Cleanup:** কিছু বদলানো হয়নি।

Lab 10 — Nginx Reverse Proxy: নিজের ওয়েবে দরজা বসানো

পরিস্থিতি: TechNova-র অ্যাপ চলে port 3000-এ, কিন্তু ব্যবহারকারী আসবে port 80-এ। Nginx reverse proxy হিসেবে সামনে বসবে।

উদ্দেশ্য: Nginx ইনস্টল করে একটি upstream-এ proxy_pass করা।

পূর্বজ্ঞান: অধ্যায় ১৭, ২১। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** port 3000-এ একটি test অ্যাপ চলছে (Lab 04-এর `nc -l 3000` বা Python-এর `python3 -m http.server 3000`)।

1. `sudo apt update && sudo apt install -y nginx` দিয়ে Nginx ইনস্টল করুন।
2. `sudo nano /etc/nginx/sites-available/reverse-proxy` খুলে লিখুন: `server { listen 80; location / { proxy_pass http://127.0.0.1:3000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } }`।
3. `sudo ln -s /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/` ও `sudo nginx -t` দিয়ে syntax যাচাই করুন।
4. `sudo systemctl reload nginx` চালিয়ে `curl -v http://127.0.0.1/` দিয়ে পরীক্ষা করুন — উত্তরে upstream-এর জবাব আসা উচিত।

```
sudo apt install -y nginx
sudo nginx -t
sudo systemctl reload nginx
curl -v http://127.0.0.1/
```

যাচাই: `nginx -t` সফল; `curl` localhost:80-এ upstream-এর জবাব ফেরত দিয়েছে। **সাধারণ ব্যর্থতা:** 502 Bad Gateway মানে upstream বন্ধ — প্রথমে `curl http://127.0.0.1:3000` দিয়ে upstream সরাসরি পরীক্ষা করুন; `nginx -t` ব্যর্থ হলে `journalctl -u nginx` দেখুন। **Cleanup:** `sudo rm /etc/nginx/sites-enabled/reverse-proxy && sudo systemctl reload nginx`।

Lab 11 — Load Balancing: দুটি Upstream-এ ট্রাফিক ভাগ করা

পরিস্থিতি: TechNova-র অ্যাপের দুটি কপি চলছে — port 3001 ও 3002-এ। Nginx দুজনের মধ্যে অনুরোধ ভাগ করবে।

উদ্দেশ্য: Nginx upstream block ও round-robin load balancing।

পূর্বজ্ঞান: অধ্যায় ১৭, Lab 10। **টুল:** VM। **প্রাথমিক অবস্থা:** Nginx ইনস্টল করা; দুটি test সার্ভার চলছে (`python3 -m http.server 3001` ও `3002` , আলাদা ডিরেক্টরিতে)।

1. upstream block লিখুন: `upstream app { server 127.0.0.1:3001; server 127.0.0.1:3002; }` এবং `proxy_pass http://app;`।
2. `sudo nginx -t && sudo systemctl reload nginx` চালান।
3. `for i in 1 2 3 4; do curl -s http://127.0.0.1/ | head -2; done` চালিয়ে দেখুন অনুরোধ দুই সার্ভারে পালা করে যাচ্ছে কি না (প্রতিটি সার্ভারের ডিরেক্টরিতে আলাদা index.html রাখলে পার্থক্য বোঝা যাবে)।
4. একটি সার্ভার বন্ধ করে আবার curl চালান — Nginx স্বয়ংক্রিয়ভাবে জীবিত সার্ভারেই পাঠায় কি না, লক্ষ করুন।

```
sudo nginx -t && sudo systemctl reload nginx
for i in 1 2 3 4; do curl -s http://127.0.0.1/; done
```

যাচাই: চারটি অনুরোধে দুই সার্ভারের জবাব পালা করে এসেছে; একটি বন্ধ থাকলেও সাইট চলেছে। **সাধারণ ভুল:** session-নির্ভর অ্যাপে round-robin-এ sticky session না থাকলে লগইন ভেঙে যেতে পারে — production-এ `ip_hash` বা shared session বিবেচনা করুন। **Cleanup:** test সার্ভার বন্ধ করুন, site ফাইল সরান, reload দিন।

Lab 12 — UFW Firewall: শুধু দরকারি দরজা খোলা

পরিস্থিতি: TechNova-র সার্ভার ইন্টারনেটে যাবে। প্রথমে firewall: SSH ও HTTP খোলা, বাকি সব বন্ধ।

উদ্দেশ্য: UFW দিয়ে allow/deny নিয়ম, সক্রিয়করণ ও যাচাই।

পূর্বজ্ঞান: অধ্যায় ১৮। **টুল:** VM (snapshot নিয়ে নিন)। **প্রাথমিক অবস্থা:** UFW নিষ্ক্রিয়।

1. `sudo ufw status` দিয়ে বর্তমান অবস্থা দেখুন।
2. `sudo ufw allow OpenSSH` চালান — **এই ধাপটি enable-এর আগে অবশ্যই করুন।**
3. `sudo ufw allow 80/tcp` ও `sudo ufw allow 443/tcp` চালান।
4. `sudo ufw enable` চালিয়ে `sudo ufw status verbose` দিয়ে নিয়মগুলো যাচাই করুন।

5. `sudo ufw deny 8080/tcp` দিয়ে একটি test port বন্ধ করে `curl http://127.0.0.1:8080/` আচরণ দেখুন (localhost-এ UFW প্রযোজ্য নাও হতে পারে — তাই বাইরের VM থেকে পরীক্ষা করাই সঠিক যাচাই)।

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

যাচাই: status-এ SSH, 80, 443 ALLOW দেখা গেছে; default deny incoming। **সাধারণ ব্যর্থতা:** SSH নিয়ম ছাড়া enable করলে নিজেই আটকে যাবেন — তখন VM console থেকে `sudo ufw disable` করতে হয়। **Cleanup:** ল্যাব শেষে `sudo ufw disable` ও নিয়মগুলো `sudo ufw reset` দিয়ে মুছে ফেলুন (নিশ্চিত হয়ে)।

Lab 13 — WireGuard VPN ধারণা: নিরাপদ টানেলের ল্যাব

পরিস্থিতি: TechNova-র কর্মী বাড়ি থেকে অফিসের private নেটওয়ার্কে পৌঁছাতে চায়। WireGuard টানেল কীভাবে কাজ করে, দুটি VM-এর মধ্যে পরীক্ষা করুন।

উদ্দেশ্য: WireGuard interface, keypair ও peer ধারণা হাতে-কলমে বোঝা।

পূর্বজ্ঞান: অধ্যায় ১৯। **টুল:** দুটি VM (peer-a, peer-b) একই Host-only নেটওয়ার্কে। **প্রাথমিক অবস্থা:** দুটি VM পরস্পরকে ping করতে পারে।

1. দুটি VM-এ `sudo apt install -y wireguard` চালান।
2. প্রতিটিতে `wg genkey | tee privatekey | wg pubkey > publickey` দিয়ে keypair তৈরি করুন — private key কখনও শেয়ার করবেন না, শুধু public key বিনিময় করুন।
3. peer-a-তে wg0 কনফিগার করুন: `[Interface] PrivateKey = <peer-a private> Address = 10.200.0.1/24 ListenPort = 51820` এবং `[Peer] PublicKey = <peer-b public> AllowedIPs = 10.200.0.2/32 Endpoint = 192.168.56.11:51820`।
4. peer-b-তে উল্টো কনফিগারেশন করুন, দুটিতে `sudo wg-quick up wg0` চালান।
5. `ping -c 4 10.200.0.2` দিয়ে টানেলের ভেতর দিয়ে পৌঁছানো যাচাই করুন; `sudo wg show` দিয়ে handshake দেখুন।

```
wg genkey | tee privatekey | wg pubkey > publickey
sudo wg-quick up wg0
sudo wg show
ping -c 4 10.200.0.2
```

যাচাই: `wg show`-এ latest handshake সময় দেখা গেছে; টানেল IP-তে ping সফল। **সাধারণ ব্যর্থতা:** Endpoint ভুল হলে handshake হয় না — `wg show`-এ handshake অনুপস্থিত থাকলে IP/port যাচাই করুন; firewall-এ UDP 51820 খুলতে হতে পারে। **নিরাপত্তা:** private key কখনও চ্যাট, ইমেইল বা Git-এ পাঠাবেন না। **Cleanup:** `sudo wg-quick down wg0` দুটি VM-এ; key ফাইল মুছে ফেলুন।

Lab 14 — tcpdump: তারের ওপর দিয়ে কী যাচ্ছে, দেখা

পরিস্থিতি: একটি অ্যাপ মাঝে মাঝে সংযোগ হারায়। Wireshark-এর আগে tcpdump দিয়ে দ্রুত দেখুন — DNS অনুরোধ যাচ্ছে কি না, TCP handshake সম্পূর্ণ হচ্ছে কি না।

উদ্দেশ্য: tcpdump filter, capture ও পড়া।

পূর্বজ্ঞান: অধ্যায় ২২। **টুল:** VM, sudo। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `sudo apt install -y tcpdump` ইনস্টল করুন।
2. একটি টার্মিনালে `sudo tcpdump -i any -n port 53 -c 10` চালান — DNS পোর্টের ১০টি প্যাকেট ধরা হবে।
3. আরেকটি টার্মিনালে `dig example.com` চালান — প্রথম টার্মিনালে DNS query/response দেখা যাবে।
4. `sudo tcpdump -i any -n host 192.168.56.1 -c 5` দিয়ে gateway-এর ট্রাফিক দেখুন।
5. `sudo tcpdump -i any -n -c 20 -w /tmp/lab14.pcap port 80` দিয়ে capture ফাইলে লিখুন; পরে `tcpdump -r /tmp/lab14.pcap` দিয়ে পড়ুন।

```
sudo tcpdump -i any -n port 53 -c 10
sudo tcpdump -i any -n -c 20 -w /tmp/lab14.pcap port 80
tcpdump -r /tmp/lab14.pcap
```

যাচাই: DNS query ও response দুটোই ধরা পড়েছে; pcap ফাইল পড়া গেছে। **সাধারণ ভুল:** `-n` ছাড়া tcpdump প্রতিটি IP-র reverse DNS করতে গিয়ে ধীর হয়। **নিরাপত্তা:** শুধু নিজের VM-এর interface-এ capture করুন; অন্যের ট্রাফিক capture করা অনুমতি ছাড়া নিষিদ্ধ। **Cleanup:** `rm /tmp/lab14.pcap`।

Lab 15 — Wireshark: TCP Handshake চোখে দেখা

পরিস্থিতি: একটি সার্ভারে সংযোগ মাঝে মাঝে ব্যর্থ হয়। Wireshark-এ handshake দেখে বুঝতে হবে — SYN যাচ্ছে, SYN-ACK ফিরছে কি না।

উদ্দেশ্য: Wireshark-এ capture, filter ও TCP handshake বিশ্লেষণ।

পূর্বজ্ঞান: অধ্যায় ১১, ২২। **টুল:** Host কম্পিউটারে Wireshark (বা VM-এ `sudo apt install -y wireshark` — GUI না থাকলে Lab 14-এর pcap Wireshark-এ খুলুন)। **প্রাথমিক অবস্থা:** একটি pcap ফাইল বা live capture।

1. Wireshark খুলে নিজের VM-এর interface বেছে capture শুরু করুন (অথবা `File → Open` দিয়ে /tmp/lab14.pcap খুলুন)।
2. Filter বারে লিখুন `tcp.flags.syn == 1` — শুধু SYN প্যাকেট দেখা যাবে।
3. একটি SYN প্যাকেটে ক্লিক করে TCP header খুলুন: Sequence Number, Window Size, MSS দেখুন।
4. Filter `tcp.stream eq 0` দিয়ে একটি পূর্ণ সংযোগের সব প্যাকেট দেখুন — SYN, SYN-ACK, ACK ক্রমটি মেলান।
5. Filter `dns` দিয়ে DNS query/response জোড়া দেখুন — Transaction ID মিলছে কি না যাচাই করুন।

```
tcp.flags.syn == 1
tcp.stream eq 0
dns
http.request
```

যাচাই: একটি stream-এ SYN → SYN-ACK → ACK ক্রমটি দেখা গেছে; DNS query-র Transaction ID response-এর সঙ্গে মিলেছে। **সাধারণ ব্যর্থতা:** SYN গিয়ে SYN-ACK না ফিরলে সার্ভার বন্ধ বা firewall ব্লক করছে; RST প্যাকেট মানে port বন্ধ। **নিরাপত্তা:** capture ফাইলে পাসওয়ার্ড বা টোকেন থাকতে পারে — ফাইলটি শেয়ার করবেন না, ল্যাব শেষে মুছে ফেলুন। **Cleanup:** capture বন্ধ করে pcap মুছুন।

Lab 16 — NAT ও Port Forwarding ধারণা: Private থেকে Public

পরিস্থিতি: TechNova-র private নেটওয়ার্কের (192.168.56.0/24) মেশিনগুলো একটি public IP দিয়ে ইন্টারনেটে যায়। NAT কীভাবে কাজ করে, তা ল্যাবে বোঝা দরকার।

উদ্দেশ্য: NAT/PAT ধারণা, `iptables`/nftables MASQUERADE পড়া ও যাচাই।

পূর্বজ্ঞান: অধ্যায় ১২। **টুল:** VM। **প্রাথমিক অবস্থা:** VM NAT মোডে ইন্টারনেটে যুক্ত।

1. `ip route` দিয়ে default gateway ও `curl -s ifconfig.me` দিয়ে বাইরে থেকে দেখা public IP (যদি ইন্টারনেট থাকে) লিখুন — দেখুন ভেতরের private IP আর বাইরের public IP আলাদা।
2. `sudo iptables -t nat -L -n -v` বা `sudo nft list ruleset` দিয়ে NAT নিয়ম আছে কি না দেখুন (VM-এ না থাকলে VirtualBox-এর NAT-ই এই কাজ করছে — সেটিই শিক্ষা: NAT একটি gateway-এর কাজ)।
3. Port forwarding ধারণা যাচাই: VirtualBox-এ VM-এর Settings → Network → Advanced → Port Forwarding-এ Host 8080 → Guest 80 নিয়ম যোগ করে host থেকে `curl http://127.0.0.1:8080/` চালান (Nginx চললে জবাব আসবে)।
4. নিয়মটি মুছে ফেলুন।

```
curl -s ifconfig.me
sudo iptables -t nat -L -n -v
ip route
```

যাচাই: ভেতরের ও বাইরের IP আলাদা দেখা গেছে; port forwarding নিয়মে host port থেকে guest সার্ভিসে পৌঁছানো গেছে। **সাধারণ ভুল:** port forwarding-কে firewall নিয়ম ভাবা — এটি ঠিকানা অনুবাদ, নিরাপত্তা নয়; forwarded port-এ সার্ভিসটি সবার কাছে খুলে যায়। **Cleanup:** forwarding নিয়ম মুছে ফেলুন।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 17 — নেটওয়ার্ক ব্যর্থতা নির্ণয়: ধাপে ধাপে, স্তরে স্তরে

পরিস্থিতি: TechNova-র সার্ভার থেকে example.com খুলছে না। কারণ কি DNS, gateway, নাকি সার্ভার নিজে? ইচ্ছাকৃতভাবে ভাঙা অবস্থা তৈরি করে স্তরে স্তরে নির্ণয় করুন।

উদ্দেশ্য: structured troubleshooting: link → IP → gateway → DNS → application।

পূর্বজ্ঞান: অধ্যায় ২২। **টুল:** VM, snapshot। **প্রাথমিক অবস্থা:** সব ঠিক আছে।

1. সব ঠিক আছে যাচাই করুন: `ping -c 2 192.168.56.1`, `ping -c 2 8.8.8.8`, `dig example.com +short`, `curl -s -o /dev/null -w "%{http_code}\n" https://example.com`।
2. ব্যর্থতা ১ তৈরি করুন: `sudo ip route del default` — এখন কী হয় দেখুন (gateway নেই)।
3. নির্ণয় করুন: `ip route` (default নেই), `ping -c 2 8.8.8.8` ব্যর্থ, কিন্তু `ping -c 2 192.168.56.1` সফল — সমস্যা gateway-এ।
4. মেরামত করুন: `sudo ip route add default via 192.168.56.1` ও আবার যাচাই করুন।
5. ব্যর্থতা ২: `/etc/resolv.conf`-এর nameserver বদলে ফেলবেন না; বরং `dig @192.0.2.1 example.com` (TEST-NET ঠিকানা, উত্তর আসবে না) দিয়ে ভুল resolver-এর আচরণ দেখুন — timeout মানে DNS পথের সমস্যা।

```
ping -c 2 192.168.56.1
ping -c 2 8.8.8.8
dig example.com +short
sudo ip route del default
sudo ip route add default via 192.168.56.1
```

যাচাই: প্রতিটি ব্যর্থতার পর কোন ধাপে আটকেছে, তা লিখতে পেরেছেন; মেরামতের পর সব পরীক্ষা আবার সফল। **সাধারণ ভুল:** একসঙ্গে অনেক কিছু বদলানো — একটি বদলান, যাচাই করুন, তারপর পরেরটি। **Cleanup:** default route ফিরিয়ে এনেছেন কি না, `ip route` দিয়ে নিশ্চিত হোন।

Lab 18 — CDN ও Cache Header: ব্রাউজার কী সংরক্ষণ করে

পরিস্থিতি: TechNova-র সাইটের ছবি বারবার ডাউনলোড হচ্ছে। Cache-Control header দেখে বুঝতে হবে — CDN/ব্রাউজার কতক্ষণ সংরক্ষণ করবে।

উদ্দেশ্য: `curl -I` দিয়ে Cache-Control, ETag, Age header পড়া।

পূর্বজ্ঞান: অধ্যায় ২০। **টুল:** VM, ইন্টারনেট। **প্রাথমিক অবস্থা:** ইন্টারনেট আছে।

1. `curl -sI https://example.com | grep -iE "cache-control|etag|age|cf-cache|server"` চালান।
2. `curl -sI https://example.com | head -20` দিয়ে পুরো header তালিকা পড়ুন।
3. নিজের Nginx-এ একটি static ফাইলের জন্য `add_header Cache-Control "public, max-age=3600";` যোগ করে `curl -I http://127.0.0.1/file.css` দিয়ে header যাচাই করুন (Lab 10-এর Nginx থাকলে)।

```
curl -sI https://example.com | grep -iE "cache-control|etag|age|server"
curl -sI https://example.com
```

যাচাই: Cache-Control/max-age মানটি পড়তে পেরেছেন; নিজের সার্ভারের header-এ max-age=3600 দেখা গেছে। **সাধারণ ভুল:** `no-cache` মানে সংরক্ষণ নিষেধ নয় — মানে ব্যবহারের আগে revalidate করতে হবে। **Cleanup:** Nginx-এর test header সরিয়ে reload দিন।

Lab 19 — SSH Tunneling: নিরাপদ পথে ভেতরের সার্ভিসে পৌঁছানো

পরিস্থিতি: TechNova-র ডেটাবেস শুধু private নেটওয়ার্কে শোনে। SSH tunnel দিয়ে নিজের মেশিন থেকে নিরাপদে পৌঁছাতে হবে, পোর্ট সবার জন্য খুলে নয়।

উদ্দেশ্য: SSH local port forwarding (`-L`) ব্যবহার করা।

পূর্বজ্ঞান: অধ্যায় ১৯। **টুল:** দুটি VM বা একটি VM + host। **প্রাথমিক অবস্থা:** SSH চলছে; target সার্ভিস শুধু 127.0.0.1:3000-এ শোনে।

1. Target VM-এ `python3 -m http.server 3000 --bind 127.0.0.1` চালান — এটি শুধু localhost থেকে পৌঁছানো যায়।
2. নিজের মেশিন থেকে `ssh -L 8080:127.0.0.1:3000 user@192.168.56.10 -N` চালান (user ও ঠিকানা আপনার ল্যাবের)।
3. আরেকটি টার্মিনালে `curl http://127.0.0.1:8080/` চালান — tunnel দিয়ে target-এর জবাব আসবে।
4. `ss -tlnp | grep 8080` দিয়ে দেখুন 8080 শুধু আপনার মেশিনে শুনছে।

```
ssh -L 8080:127.0.0.1:3000 user@192.168.56.10 -N
curl http://127.0.0.1:8080/
ss -tlnp | grep 8080
```

যাচাই: tunnel ছাড়া 3000-এ পৌঁছানো যায় না, tunnel দিয়ে 8080-এ জবাব আসে। **সাধারণ ব্যর্থতা:** `channel 3: open failed: connect failed` মানে target সার্ভিস বন্ধ — প্রথমে target-এ সরাসরি পরীক্ষা করুন। **নিরাপত্তা:** tunnel ব্যবহার শেষে Ctrl+C দিয়ে বন্ধ করুন; `-N` মানে shell নয়, শুধু forwarding। **Cleanup:** tunnel ও test সার্ভার বন্ধ করুন।

Lab 20 — IPv6: Link-Local থেকে Global

পরিস্থিতি: TechNova-র নতুন নেটওয়ার্কে IPv6 চালু হচ্ছে। আপনার VM-এ IPv6 ঠিকানা আছে কি, link-local কীভাবে কাজ করে — দেখুন।

উদ্দেশ্য: IPv6 ঠিকানা পড়া, সংক্ষেপণ ও ping6।

পূর্বজ্ঞান: অধ্যায় ৯। **টুল:** VM। **প্রাথমিক অবস্থা:** IPv6 সক্রিয় (ডিফল্ট)।

1. `ip -6 addr show` চালান — fe80:: দিয়ে শুরু link-local ও global ঠিকানা আলাদা করুন।
2. `ping -6 -c 3 fe80::1%enp0s3` চালিয়ে দেখুন link-local-এ interface উল্লেখ (%enp0s3) কেন দরকার (interface নাম আপনার VM-এর)।
3. `ip -6 route show` দিয়ে IPv6 default route আছে কি না দেখুন।

4. `dig example.com AAAA +short` দিয়ে একটি সাইটের IPv6 ঠিকানা নিন; `ping -6 -c 2` দিয়ে পরীক্ষা করুন (ইন্টারনেটে IPv6 থাকলে)।

```
ip -6 addr show
ip -6 route show
dig example.com AAAA +short
```

যাচাই: link-local (fe80::/10) ও global ঠিকানা আলাদা করতে পেরেছেন; `::` সংক্ষেপণের উদাহরণ লিখেছেন। **সাধারণ ভুল:** link-local ঠিকানায় scope/zone (%) ছাড়া ping করা — কোন interface দিয়ে যাবে, তা OS জানে না। **Cleanup:** কিছু বদলানো হয়নি।

Lab 21 — DHCP Lease যাচাই ও Static IP দেওয়া (ল্যাব)

পরিস্থিতি: TechNova-র একটি সার্ভারকে স্থায়ী IP দিতে হবে — DHCP-তে ভরসা নয়। Netplan দিয়ে static IP কনফিগার করুন, তবে ভুল হলে ফেরার পথ রাখুন।

উদ্দেশ্য: Netplan YAML দিয়ে static address, gateway ও DNS দেওয়া।

পূর্বজ্ঞান: অধ্যায় ১৪, ২১। **টুল:** VM, snapshot (অবশ্যই নিন)। **প্রাথমিক অবস্থা:** DHCP-তে ঠিকানা পাওয়া।

1. `ls /etc/netplan/` দিয়ে বর্তমান ফাইল দেখুন; `sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak` দিয়ে ব্যাকআপ নিন (ফাইলের নাম আপনার VM-এ ভিন্ন হতে পারে)।
2. ফাইলটি সম্পাদনা করে DHCP-র বদলে লিখুন: `dhcp4: false`, `addresses: [192.168.56.20/24]`, `routes: [{to: default, via: 192.168.56.1}]`, `nameservers: {addresses: [8.8.8.8, 1.1.1.1]}` — YAML-এ indentation ঠিক রাখুন (space, tab নয়)।
3. `sudo netplan try` চালান — ১২০ সেকেন্ডের মধ্যে নিশ্চিত না করলে পুরোনো অবস্থায় ফিরে যাবে; এটাই নিরাপত্তা।
4. নিশ্চিত করে `sudo netplan apply` চালিয়ে `ip addr show` ও `ping -c 2 8.8.8.8` দিয়ে যাচাই করুন।

```
sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-cloud-init.yaml.bak
sudo netplan try
sudo netplan apply
ip addr show
```

যাচাই: static IP দেখা গেছে; gateway ও DNS কাজ করছে। **সাধারণ ব্যর্থতা:** YAML indentation ভুল হলে `netplan try` ভুল জানাবে — tab ব্যবহার করবেন না। **Cleanup:** ব্যাকআপ ফাইল থেকে ফিরতে `sudo cp /etc/netplan/50-cloud-init.yaml.bak /etc/netplan/50-cloud-init.yaml && sudo netplan apply`।

Lab 22 — VLAN ধারণা: এক সুইচে দুটি আলাদা নেটওয়ার্ক

পরিস্থিতি: TechNova-র অফিস ও guest নেটওয়ার্ক একই সুইচে, কিন্তু আলাদা থাকতে হবে। VLAN ট্যাগিং কীভাবে আলাদা করে, তা ল্যাবে বোঝা দরকার।

উদ্দেশ্য: Linux-এ VLAN interface তৈরি ও যাচাই (ধারণাগত ল্যাব)।

পূর্বজ্ঞান: অধ্যায় ৬, ১৮। **টুল:** VM। **প্রাথমিক অবস্থা:** একটি interface।

1. `sudo apt install -y vlan` ইনস্টল করুন (প্রয়োজন না হলেও টুলটি চেনা ভালো)।

2. `sudo ip link add link enp0s3 name enp0s3.10 type vlan id 10` চালান — VLAN 10-এর virtual interface তৈরি হলো।
3. `sudo ip addr add 192.168.10.10/24 dev enp0s3.10 && sudo ip link set enp0s3.10 up` চালান।
4. `ip -d link show enp0s3.10` দিয়ে VLAN ID ও `ip addr show enp0s3.10` দিয়ে ঠিকানা যাচাই করুন।
5. `sudo ip link del enp0s3.10` দিয়ে মুছে ফেলুন।

```
sudo ip link add link enp0s3 name enp0s3.10 type vlan id 10
sudo ip addr add 192.168.10.10/24 dev enp0s3.10
ip -d link show enp0s3.10
sudo ip link del enp0s3.10
```

যাচাই: VLAN interface-এ id 10 দেখা গেছে; মুছে ফেলার পর আর নেই। **মনে রাখবেন:** আসল VLAN আলাদা করতে সুইচে trunk/access পোর্ট কনফিগার করতে হয় — শুধু VM-এ interface বানালেই নেটওয়ার্ক আলাদা হয় না। **Cleanup:** interface মুছে ফেলা হয়েছে।

Lab 23 — Cloud Network Architecture: কাগজে VPC ডিজাইন

পরিস্থিতি: TechNova-র অ্যাপ cloud-এ যাবে। কোনো খরচ ছাড়াই, কাগজে একটি VPC ডিজাইন করুন — public/private subnet, route table, NAT gateway, security group।

উদ্দেশ্য: cloud নেটওয়ার্কের উপাদানগুলো একত্রে সাজানো (ডিজাইন ল্যাব, কোনো cloud অ্যাকাউন্ট দরকার নেই)।

পূর্বজ্ঞান: অধ্যায় ২৩, ২৪। **টুল:** কাগজ বা draw.io। **প্রাথমিক অবস্থা:** CIDR 10.0.0.0/16 বরাদ্দ।

1. 10.0.0.0/16-কে ভাগ করুন: public 10.0.1.0/24 ও 10.0.2.0/24 (দুটি availability zone), private 10.0.11.0/24 ও 10.0.12.0/24, database 10.0.21.0/24 ও 10.0.22.0/24।
2. Internet gateway public subnet-এর route table-এ (0.0.0.0/0 → IGW); NAT gateway public subnet-এ বসিয়ে private subnet-এর route table-এ (0.0.0.0/0 → NAT)।
3. Security group লিখুন: ALB-এ 80/443 সবার থেকে; app সার্ভারে শুধু ALB থেকে; database-এ শুধু app subnet থেকে 5432।
4. ডায়াগ্রামে প্রতিটি subnet, route ও security group-এর নিয়ম লেবেল করুন।

ডায়াগ্রাম — VPC 10.0.0.0/16

Internet → Internet Gateway → Public subnet (ALB, NAT GW) → Private subnet (App) → DB subnet (Database, শুধু App থেকে 5432)

Public route: 0.0.0.0/0 → IGW | Private route: 0.0.0.0/0 → NAT GW

যাচাই: প্রতিটি subnet-এর CIDR ওভারল্যাপ করে না; DB subnet-এ কোনো internet route নেই; প্রতিটি tier-এর নিয়ম লেখা আছে। **সাধারণ ভুল:** database subnet-কে public করা; NAT gateway-কে private subnet-এ বসানো (সেটি কাজ করবে না, NAT-কে public-এ থাকতে হয়)। **Cleanup:** এটি ডিজাইন ল্যাব — কোনো resource তৈরি হয়নি, খরচ শূন্য।

Lab 24 — Multi-Tier Network Health Check: পুরো বইয়ের সমন্বয়

পরিস্থিতি: TechNova-র তিন-স্তরের (web/app/db) ল্যাব নেটওয়ার্কের সাপ্তাহিক স্বাস্থ্য পরীক্ষা। একটি স্ক্রিপ্টে সব যাচাই একত্র করুন।

উদ্দেশ্য: interface, route, DNS, port, HTTP ও TLS যাচাইয়ের একক চেকলিস্ট চালানো।

পূর্বজ্ঞান: অধ্যায় ২১–২৪, Lab 01–23। **টুল:** VM, Nginx (Lab 10)। **প্রাথমিক অবস্থা:** Nginx চলছে।

1. একটি চেকলিস্ট ফাইল তৈরি করুন: `ip -brief addr`, `ip route`, `dig example.com +short`, `ss -tlnp`, `curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/`, `sudo ufw status`।
2. প্রতিটি কমান্ড চালিয়ে ফলাফল লিখুন — কোনটি সফল, কোনটি ব্যর্থ, কেন।
3. ইচ্ছাকৃতভাবে একটি ব্যর্থতা তৈরি করুন (Nginx বন্ধ করুন: `sudo systemctl stop nginx`), চেকলিস্ট আবার চালিয়ে কোন ধাপে ধরা পড়ল, তা লিখুন।
4. `sudo systemctl start nginx` দিয়ে ফিরিয়ে এনে শেষ যাচাই করুন।

```
ip -brief addr; ip route
dig example.com +short
ss -tlnp
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/
sudo ufw status
```

যাচাই: Nginx বন্ধ থাকলে curl ধাপে ব্যর্থতা ধরা পড়েছে; চালু থাকলে সব ধাপ সবুজ। **সাধারণ ভুল:** একটি ধাপের ব্যর্থতাকে পুরো নেটওয়ার্কের ব্যর্থতা ভাবা — ধাপটি লিখুন, তারপর কারণ খুঁজুন। **Cleanup:** Nginx চালু অবস্থায় রেখে যান; চেকলিস্ট ফাইলটি সংরক্ষণ করুন — এটি আপনার প্রথম runbook।

পরিশিষ্ট B — Independent Assignments (স্বাধীন অ্যাসাইনমেন্ট)

নিচের ১৫টি অ্যাসাইনমেন্টের সমাধান এই বইয়ে দেওয়া নেই — সমাধান-নির্দেশিকা আলাদা ভলিউমে আছে। প্রতিটি অ্যাসাইনমেন্ট করার আগে সংশ্লিষ্ট অধ্যায় ও ল্যাব শেষ করুন। উত্তর লেখার সময় শুধু ফলাফল নয়, পদ্ধতি ও যুক্তিও লিখুন।

Beginner — অ্যাসাইনমেন্ট ১–৫

1. **নিজের নেটওয়ার্কের মানচিত্র:** আপনার VM-এর `ip addr`, `ip route` ও `ip neigh` আউটপুট থেকে একটি ডায়াগ্রাম আঁকুন — interface, IP, gateway, DNS resolver ও প্রতিবেশী MAC-সহ। প্রতিটি উপাদানের কাজ এক লাইনে লিখুন।
2. **Binary অনুশীলন:** 192.168.10.25, 255.255.255.0 ও 172.16.0.1-কে binary-তে রূপান্তর করুন; 11000000.10101000.00001010.00011001-কে decimal-এ ফেরত নিন। প্রতিটি ধাপ দেখান।
3. **OSI-তে একটি অনুরোধ:** ব্রাউজারে example.com খোলার যাত্রা OSI-র সাতটি স্তরে লিখুন — প্রতিটি স্তরে কী যোগ হয়, কোন ডিভাইস কাজ করে।
4. **Port শনাক্তকরণ:** আপনার VM-এ `ss -tlnp` চালিয়ে প্রতিটি listening port-এর সার্ভিস, process ও ঝুঁকি লিখুন। কোন port-টি বাইরে থেকে পৌঁছানো উচিত নয়, এবং কেন?
5. **DNS তদন্ত:** আপনার পছন্দের একটি ডোমেইনের A, AAAA, MX ও NS রেকর্ড `dig` দিয়ে বের করুন। TTL মানগুলো লিখে ব্যাখ্যা করুন — TTL বদলালে propagation-এ কী প্রভাব পড়ে?

Subnetting ও গণনা — অ্যাসাইনমেন্ট ৬–৮

1. **Subnet ভাগ:** 10.10.0.0/22 নেটওয়ার্ককে চারটি সমান subnet-এ ভাগ করুন। প্রতিটির network, broadcast, usable range ও host সংখ্যা লিখুন। তারপর একটি /23 subnet-এর জন্য একই হিসাব করুন।
2. **VLSM পরিকল্পনা:** একটি অফিসে চারটি বিভাগের প্রয়োজন — 100, 50, 25 ও 10টি host। 192.168.40.0/24 থেকে VLSM দিয়ে প্রতিটি বিভাগের subnet বরাদ্দ করুন, যাতে কোনো ঠিকানা অপচয় না হয়।
3. **CIDR যাচাই:** 172.16.8.0/21-এর network, broadcast ও usable host সংখ্যা বের করুন। 172.16.15.255 ঠিকানাটি এই নেটওয়ার্কের ভেতরে কি না, যুক্তি দিয়ে দেখান।

Protocol ও HTTP — অ্যাসাইনমেন্ট ৯–১১

1. **TCP বনাম UDP সিদ্ধান্ত:** ভিডিও কল, ফাইল ডাউনলোড, DNS lookup ও অনলাইন গেম — প্রতিটির জন্য TCP না UDP, যুক্তিসহ লিখুন। একটি ক্ষেত্রে ভুল প্রোটোকল বেছে নিলে কী ক্ষতি হবে?
2. **HTTP পরিদর্শন:** `curl -v` দিয়ে একটি সাইটের GET অনুরোধের request header ও response header লিখুন। প্রতিটি header-এর কাজ ব্যাখ্যা করুন; 301 ও 302-এর পার্থক্য লিখুন।
3. **TLS যাচাই:** `openssl s_client` দিয়ে দুটি সাইটের certificate-এর issuer, মেয়াদ ও TLS সংস্করণ তুলনা করুন। একটি certificate মেয়াদ শেষ হলে ব্যবহারকারী কী দেখবে, তা লিখুন।

ডিজাইন ও Troubleshooting — অ্যাসাইনমেন্ট ১২–১৫

1. **Firewall নীতি ডিজাইন:** একটি web সার্ভারের জন্য UFW নীতি লিখুন — SSH (শুধু অফিস IP থেকে), HTTP/HTTPS (সবার থেকে), বাকি সব বন্ধ। প্রতিটি নিয়মের যুক্তি ও ভুলের ঝুঁকি লিখুন।
2. **নেটওয়ার্ক ডায়াগ্রাম:** একটি ছোট স্টার্টআপের নেটওয়ার্ক ডিজাইন করুন — router, switch, firewall, web/app/db সার্ভার, VLAN। IP পরিকল্পনাসহ ডায়াগ্রাম আঁকুন ও প্রতিটি সংযোগে প্রোটোকল লিখুন।
3. **Troubleshooting কেস:** "সার্ভার থেকে ping যায়, কিন্তু curl ব্যর্থ" — সম্ভাব্য পাঁচটি কারণ লিখুন এবং প্রতিটি যাচাইয়ের কমান্ড দিন। কোন কারণটি সবচেয়ে সম্ভাব্য, যুক্তি দিন।
4. **Production স্থাপত্য:** TechNova-র অ্যাপের জন্য একটি পূর্ণ network স্থাপত্য প্রস্তাব লিখুন — public/private segmentation, DNS, TLS, reverse proxy, load balancing, firewall, DB isolation, monitoring। প্রতিটি সিদ্ধান্তের trade-off (খরচ, জটিলতা, নিরাপত্তা) এক অনুচ্ছেদে লিখুন। সঙ্গে Lab 17-এর ব্যর্থতা পরীক্ষার ভিত্তিতে একটি এক পৃষ্ঠার incident রিপোর্ট যোগ করুন — লক্ষণ, নির্ণয়ের ধাপ, মূল কারণ, সমাধান, ভবিষ্যতে প্রতিরোধ।

পরিশিষ্ট C — Major Projects (তিনটি বড় প্রজেক্ট)

Project 01 — Network Investigation Toolkit

ব্যবসায়িক পরিস্থিতি: TechNova-র সাপোর্ট দল প্রতিদিন একই প্রশ্ন পায় — "সার্ভার কি নেটওয়ার্কে আছে?" প্রতিবার হাতে হাতে কমান্ড চালানো ধীর ও ভুলপ্রবণ। আপনার কাজ: একটি Linux diagnostic toolkit তৈরি করা, যা এক কমান্ডে interface, IP, route, DNS, listening port ও connectivity যাচাই করে একটি প্রতিবেদন দেয়।

প্রয়োজনীয়তা: Bash স্ক্রিপ্ট; `ip`, `ss`, `dig`, `ping`, `curl` ব্যবহার; প্রতিটি যাচাইয়ের ফলাফল PASS/FAIL; আউটপুট ফাইলে সংরক্ষণ; কোনো credential বা সংবেদনশীল তথ্য আউটপুটে নয়।

আর্কিটেকচার — Toolkit

toolkit.sh → ip addr/route → ss -tlnp → dig + ping + curl → প্রতিবেদন (report.txt)

প্রতিটি ধাপ স্বাধীন; একটি ধাপ ব্যর্থ হলেও বাকিগুলো চলবে।

মাইলফলক: (১) যাচাইয়ের তালিকা লেখা; (২) স্ক্রিপ্ট লেখা ও VM-এ চালানো; (৩) ইচ্ছাকৃত ব্যর্থতা (Nginx বন্ধ, DNS ভুল) দিয়ে পরীক্ষা; (৪) সীমাবদ্ধতা নথিভুক্ত করা।

ডেলিভারেবল: toolkit.sh, নমুনা প্রতিবেদন, README (ব্যবহার ও সীমাবদ্ধতা)। **গ্রহণযোগ্যতা:** সব যাচাই চলে; ব্যর্থতা সঠিকভাবে FAIL দেখায়; স্ক্রিপ্ট শুধু পড়ার কাজ করে, কোনো সার্ভিস বদলায় না।

মানদণ্ড	ভালো	উত্তম
Coverage	৫টি যাচাই আছে	৮+ যাচাই, প্রতিটির ব্যাখ্যা আছে
নির্ভরযোগ্যতা	স্বাভাবিক অবস্থায় চলে	ব্যর্থ অবস্থাতেও থামে না, স্পষ্ট বার্তা দেয়
নথি	ব্যবহার লেখা আছে	সীমাবদ্ধতা ও ভুল-ধারণা লেখা আছে

উন্নত চ্যালেঞ্জ: JSON আউটপুট যোগ করুন; একটি ধাপে latency মাপুন; cron-এ সাপ্তাহিক চালানোর পরিকল্পনা লিখুন।

Project 02 — Secure Web Application Network

ব্যবসায়িক পরিস্থিতি: TechNova-র web অ্যাপ সরাসরি port 3000-এ চলছে — কোনো TLS নেই, firewall নেই, ডোমেইন নেই। আপনার কাজ: Nginx-এর পেছনে অ্যাপটি সুরক্ষিতভাবে প্রকাশ করা।

প্রয়োজনীয়তা: DNS রেকর্ড পরিকল্পনা (A/AAAA), TLS (self-signed নয় — ল্যাবে Let's Encrypt ধারণা বা locally trusted certificate-এর পরিকল্পনা; আসল ডোমেইন না থাকলে self-signed শুধু ল্যাবে, সীমাবদ্ধতাসহ), Nginx reverse proxy, UFW firewall, upstream health যাচাই, verification রিপোর্ট।

আর্কিটেকচার — Secure Web App

Internet → DNS → Firewall (80/443) → Nginx (TLS termination, reverse proxy) → App 127.0.0.1:3000

SSH শুধু অফিস নেটওয়ার্ক থেকে; DB থাকলে শুধু App থেকে।

মাইলফলক: (১) DNS ও TLS পরিকল্পনা; (২) Nginx reverse proxy (Lab 10); (৩) TLS চালু ও `openssl s_client` দিয়ে যাচাই; (৪) UFW নিয়ম ও বাইরে থেকে যাচাই; (৫) verification রিপোর্ট।

ডেলিভারেবল: নেটওয়ার্ক ডায়াগ্রাম, Nginx কনফিগারেশন, UFW নিয়মের তালিকা, TLS যাচাইয়ের আউটপুট, verification রিপোর্ট। **গ্রহণযোগ্যতা:** HTTP থেকে HTTPS-এ redirect; TLS মেয়াদ ও chain যাচাই সফল; firewall-এ শুধু প্রয়োজনীয় port খোলা; upstream বন্ধ হলে 502-এর বদলে অর্থবোধক error পৃষ্ঠা।

মানদণ্ড	ভালো	উত্তম
TLS	HTTPS চলে	chain, মেয়াদ, redirect যাচাই নথিভুক্ত
Firewall	80/443 খোলা	SSH সীমিত, বাকি সব বন্ধ, যাচাই আছে
যাচাই	curl দিয়ে পরীক্ষা	ব্যর্থতার পরীক্ষা (upstream বন্ধ, ভুল পথ) নথিভুক্ত

উন্নত চ্যালেঞ্জ: rate limiting যোগ করুন; দুটি upstream দিয়ে load balancing যোগ করুন; security header (HSTS) পরিকল্পনা লিখুন।

Project 03 — TechNova Production Network Capstone

ব্যবসায়িক পরিস্থিতি: TechNova একটি fictional সফটওয়্যার কোম্পানি। তাদের web অ্যাপে তিনটি tier — web, application, database। বর্তমানে সবকিছু এক নেটওয়ার্কে, কোনো segmentation নেই। আপনি জুনিয়র নেটওয়ার্ক ইঞ্জিনিয়ার হিসেবে production নেটওয়ার্কের পূর্ণ নকশা ও ল্যাব প্রদর্শন করবেন।

প্রয়োজনীয়তা: Public/private segmentation, IP ও subnet পরিকল্পনা, routing ও gateway, DNS, TLS, reverse proxy, load balancing, firewall নীতি, database isolation, troubleshooting পরিকল্পনা, availability, monitoring ও নথিপত্র।

আর্কিটেকচার — TechNova Production Network

Internet → Firewall → Public subnet (Load Balancer / Nginx) → Private subnet (App tier) → DB subnet (Database, শুধু App থেকে 5432)

DNS: public zone + private zone | Monitoring: প্রতিটি tier-এর health check | Backup: DB subnet থেকে আলাদা নেটওয়ার্কে

মাইলফলক: (১) প্রয়োজন সংগ্রহ ও IP পরিকল্পনা (VLSM-সহ); (২) ডায়াগ্রাম ও firewall নীতি; (৩) ল্যাবে Nginx reverse proxy + load balancing (Lab 10–11); (৪) DNS ও TLS পরিকল্পনা; (৫) ব্যর্থতার পরীক্ষা — একটি app সার্ভার বন্ধ, একটি subnet-এর route মুছে ফেলা, DNS ভুল — প্রতিটির নির্ণয় নথিভুক্ত; (৬) চূড়ান্ত প্রতিবেদন।

ডেলিভারেবল: স্থাপত্য নথি (ডায়াগ্রামসহ), IP/subnet পরিকল্পনার টেবিল, firewall নীতি, DNS রেকর্ড তালিকা, TLS পরিকল্পনা, ল্যাব যাচাইয়ের প্রতিবেদন, incident response চেকলিস্ট, monitoring পরিকল্পনা।

গ্রহণযোগ্যতা: DB subnet ইন্টারনেট থেকে পৌঁছানো যায় না; প্রতিটি tier-এর নিয়ম লেখা ও যাচাই করা; একটি app সার্ভার বন্ধ হলেও সাইট চলে; প্রতিটি ব্যর্থতার নির্ণয়ের ধাপ নথিভুক্ত।

মানদণ্ড	ভালো	উত্তম
Segmentation	তিনটি tier আলাদা subnet-এ	প্রতিটি tier-এর firewall নিয়ম যাচাইসহ

Availability	একটি সার্ভার বন্ধ হলেও চলে	failover পরীক্ষার প্রমাণ আছে
নথি	ডায়াগ্রাম ও IP তালিকা আছে	runbook, incident চেকলিস্ট ও monitoring আছে

উন্নত চ্যালেঞ্জ: দ্বিতীয় availability zone-এর পরিকল্পনা যোগ করুন; VPN দিয়ে admin access-এর নকশা করুন; খরচের হিসাব (cloud-এ) লিখুন — কোন উপাদানে কত খরচ, তা অনুমানসহ।